

LIVING WORLD GAME DESIGN DOCUMENT

Persistent Agent World · Master Strategy RPG · Simulation

게임 기획 및 기술 설계 통합 문서

UTF-8 텍스트 기반 / 한국어 PDF

1. 게임 개요

플레이어가 Master가 되어 다수의 캐릭터와 조직을 관리하면서 시간이 흐르는 지속형 세계를 탐험 · 공략 · 확장하는 온라인 전략 RPG다. 핵심은 AI와 대화하는 것이 아니라 캐릭터와 세계가 장기적인 상태를 유지한다는 데 있다.

2. 핵심 철학

Persistent Identity, Autonomous Agency, Emergent Narrative, Meaningful Consequence, Hierarchical Simulation, Cost - Aware AI를 핵심 설계 원칙으로 한다.

3. Master 시스템

Master는 신이 아니라 목표와 제약조건을 전달하는 전략적 상위 주체다. 캐릭터 영입, 파티 편성, 임무 지정, 정책 설정, 자원 관리, 관계 관리, 세력 및 지역에 대한 제한적 영향력을 가진다.

4. Character Agent

각 핵심 캐릭터는 Identity, Personality, Memory, Relationship, Emotion, Needs, Motivation, Goals, Belief, Knowledge, Experience, Physical State를 가진다. 캐릭터의 행동은 이 상태의 지속적인 상호작용에서 나온다.

5. Self Model

Self Model은 캐릭터가 자신에 대해 가진 내부 표현이다. 나는 누구인가, 무엇을 원하는가, 무엇을 두려워하는가, 누구를 믿는가, 무엇을 기억하는가가 행동 결정의 입력이 된다.

6. Memory

Episodic, Semantic, Social, Emotional, Procedural Memory를 분리한다. 모든 대화를 무제한 저장하지 않고 사건을 정규화 · 요약 · 압축한다. 검색은 최근성, 중요도, 관련성, 감정 강도, 관계 가중치를 사용한다.

7. Decision Architecture

L0 Rule/State Machine → L1 Utility AI → L2 GOAP/Planner → L3 Tactical Planner → L4 ML/LLM 순서의 계층형 의사결정을 사용한다. LLM은 게임 상태의 진실 원천이 아니다.

8. World Simulation

플레이어 주변은 High Fidelity, 같은 지역은 Medium, 먼 지역은 Low, 장기 역사는 Abstract Simulation으로 처리한다. 모든 NPC를 매 프레임 Actor로 유지하지 않는다.

9. World State

지형, 인구, 세력, 경제, 정치, 생태, 기술, 역사, 시간 · 날씨가 서로 영향을 준다. NPC 행동이 세계를 변화시키고 세계 변화가 다시 NPC의 목표와 행동을 바꾼다.

10. Event System

CharacterRescued, CharacterDied, FactionDeclaredWar, CityDestroyed, MarketCrash, QuestCompleted 등의 이벤트를 공통 데이터 구조로 기록한다. Event는 Memory, Relationship, Quest, History, Analytics에서 재사용한다.

11. Relationship

Trust, Affection, Fear, Respect, Hatred, Debt, Loyalty, Familiarity의 다차원 관계 모델을 사용한다. 관계는 행동 확률을 변화시키지만 행동을 강제로 결정하지 않는다.

12. Death / Legacy

죽음은 단순한 리스폰이 아니라 세계 이벤트다. 재산, 장비, 명성, 관계, 기록 일부가 세계에 남는다. 후계자는 독립적인 Identity를 가지면서 과거의 흔적을 일부 상속받을 수 있다.

13. Progression

캐릭터는 능력 · 숙련 · 장비 · 직업 · 관계 · 지식 · 명성을 성장시킨다. Master는 정보 · 영향력 · 지휘 범위 · 조직 운영 능력을 성장시킨다. 성장의 핵심은 행동 공간의 확장이다.

14. Economy

자원 → 생산 → 유통 → 소비 → 공급/수요 → 가격의 인과관계를 구현한다. 전쟁, 탐험, 자원 발견, 플레이어 행동이 지역 경제와 NPC 행동에 영향을 준다.

15. Faction

세력은 목표와 자원을 가진 집단 에이전트다. 영토, 자금, 병력, 정보, 정책, 외교관계, 내부 파벌, 승계를 관리한다.

16. Quest / Content

퀘스트는 NPC Goal, Faction Conflict, Economic Event, World Discovery, Player Action, Historical Event에서 파생될 수 있다. 생성 콘텐츠도 구조화된 데이터와 서버 검증을 거친다.

17. Expedition / Dungeon

던전은 고정된 반복 스테이지가 아니라 세계의 구성요소다. 정찰, 탐험 실패, 자원 발견, 보스 상태, 지역 경제가 서로 연결된다.

18. Combat AI

즉시 회피 · 타깃 선정은 Rule/Utility, 진형과 복합 작전은 Tactical Planner/GOAP를 사용한다. 동료 구조 여부는 관계 · 목표 · 위험도 · 현재 상태를 함께 고려한다.

19. LLM 정책

대화 표현, 중요 NPC의 복합 계획, 장기 목표 재평가, 기억 요약, 퀘스트 서술 등에 선택적으로 사용한다. 금전 · 아이템 · 권한 · 피해량 · 사망 · 보상 등 핵심 상태를 직접 변경할 권한은 주지 않는다.

20. AI 비용

NPC 수가 아니라 고비용 추론이 필요한 사건 수에 비례하도록 설계한다. 10,000 NPC가 있어도 10,000개의 LLM을 상시 실행하지 않는다.

21. 서버 구조

클라이언트는 입력과 표현, Dedicated Game Server는 실시간 게임 상태, Backend는 계정 · 저장 · 가차 · 결제 · 친구 · 길드 등을 담당한다. 서버가 게임 상태의 권위를 가진다.

22. 기술 스택

게임 클라이언트/게임 서버는 Unreal Engine 5 + C++/Blueprint, Backend는 TypeScript 또는 C++, AI/ML은 Python, DB는 PostgreSQL, Cache는 Redis, 분석은 ClickHouse 계열을 권장한다.

23. Lobby / UI

게임 시작 → 인증 → 프로필 로드 → Lobby. Lobby에서는 캐릭터 · 가차 · 장비 · Master · 친구 · 매칭 · 원정을 관리한다. UI는 UE5의 UMG/Common UI 계층에서 별도로 구성한다.

24. Game Start Flow

Mission 선택 → Matchmaking → Game Server 할당 → World 로드 → Character State 로드 → 실제 Character Actor Spawn → Gameplay. 종료 후 Event · Memory · Relationship · Experience를 저장하고 Lobby로 복귀한다.

25. Data / Asset

Skeletal Mesh, Animation, VFX, Sound 등의 Physical Asset과 Identity, Personality, Goal, AI Profile 등의 Data Asset을 분리한다. Character Record, Character Actor, Agent도 서로 분리한다.

26. World / Map

대규모 월드는 World Partition을 사용하고 Data Layer로 기본 세계, 던전, 퀘스트 상태, 전쟁 상태 등을 분리한다. 추상 NPC를 플레이어 접근 시 Physical Actor로 전환한다.

27. Gacha / Character Generation

가차는 완성된 캐릭터 에셋을 단순히 뽑는 기능이 아니라 Character Seed에서 독립적인 Character Instance를 생성하는 시스템으로 설계한다.

28. Autonomous Growth

Experience → Behavior → Outcome → Learning → State Change → Growth 순환을 만든다. 성장에는 능력치뿐 아니라 전술 선호, 위험 회피, 관계, 목표, 숙련, 행동 정책 변화가 포함된다.

29. MVP

작은 지역 1개, NPC 50~100명, 핵심 캐릭터 10~20명, Master 1명, 던전 1개, Memory, Relationship, Utility AI, GOAP, Event System, 간단한 경제, 영구 사망, 기본 원정을 구현한다.

30. 첫 Vertical Slice

실제 캐릭터 에셋 하나가 Spawn되고 이동 · 감지 · 기억 · 목표 · 판단 · 전투 · 부상 · 퇴각 · 경험 · 성장까지 수행한 뒤 DB에 저장되고 재접속 후 동일한 존재로 복원되는 것을 첫 번째 성공 기준으로 삼는다.

31. 개발 순서

UE5 C++ 프로젝트 → Git → Core/Agent/World/Gameplay 경계 → 실제 에셋 Import → CharacterDefinition → Character State → Agent → Memory → Personality → Goal → Utility AI → GOAP → Event → DB Persistence → Lobby → Dedicated Server → 자율성장 → 가차 → 선택적 LLM 순으로 진행한다.

32. 성공 지표

NPC 장기 일관성, Player Authorship, Emergent Event Rate, Memory Relevance, Simulation Cost, LLM Cost per Active Player, Retention, Death Meaningfulness를 핵심 지표로 측정한다.

33. 주요 위험

AI 비용 폭발, NPC 행동의 부자연스러움, 세계 복잡도 폭발, 콘텐츠 부족, 기술 데모화, LLM 환각, 서버 비용 증가를 주요 위험으로 보고 계층형 AI, 이벤트 기반 호출, Server Validation, Simulation Budget, MVP 범위 제한으로 대응한다.

34. IP 원칙

독자적인 세계관 · 명칭 · 캐릭터 · 지역 · 스토리 · 수치 · 아트 스타일을 구축한다. 특정 기존 작품의 고유 캐릭터 · 고유명사 · 세계관 · 스토리를 복제하지 않는다.

35. 최종 설계 명제

이 게임의 경쟁력은 AI NPC가 말을 잘하는 것이 아니다. 캐릭터가 지속적인 상태를 가지고, 그 상태가 행동을 만들고, 행동이 세계를 변화시키며, 그 변화가 다시 기억과 목표를 바꾸는 페루프가 핵심이다.

36. 전체 시스템 흐름

PLAYER → LOGIN → PROFILE → LOBBY → MASTER → MISSION → MATCHMAKING → GAME SERVER →
WORLD
WORLD → CHARACTER ACTOR + AGENT → SELF MODEL → MEMORY / GOAL / RELATIONSHIP →
DECISION → ACTION → EVENT
EVENT → WORLD STATE + MEMORY + EXPERIENCE + HISTORY → DATABASE → 재접속 후 복원

37. 최종 목표

플레이어가 '내가 없는 동안에도 세계가 살아 있었다', '이 캐릭터는 내가 어제 만난 그 캐릭터다', '내가 만든 사건이 오래 뒤에도 세계에 남아 있다'고 느끼게 만드는 것이 최종 목표다.